

PREDICTING ONLINE SHOPPERS' PURCHASE INTENTION

**RENIER P. APAL
RODNEE GLEN A. MARTIN
EARL LENSER B. BOLANSOY**

**A CAPSTONE COMPUTATIONAL SCIENCE PROJECT IN PARTIAL
FULFILLMENT OF THE FINAL REQUIREMENTS FOR THE COURSE IN
COMPUTATIONAL SCIENCE**

BACHELOR OF SCIENCE IN COMPUTER SCIENCE



MAY 2026

ABSTRACT

This capstone investigates whether a k-Nearest Neighbors (kNN) classifier, implemented from first principles in NumPy, can predict online purchase intent from session-level web analytics. Using the UCI Online Shoppers Purchasing Intention dataset (12,330 sessions, 17 features, 15.47% positive class), the researchers built a complete pipeline covering exploratory analysis, one-hot encoding, stratified train/test splitting, train-only standardization, hand-coded kNN with majority-vote prediction, and exhaustive tuning of k from 1 to 30. The final model ($k = 7$, Euclidean distance) achieved 87.67% accuracy, 0.6974 precision, 0.3570 recall, and 0.4722 F1 on a held-out test set, validated by 5-fold stratified cross-validation. The model outperformed both a majority-class baseline (zero recall) and a single-feature rule on PageValues. The dominant predictors were PageValues, BounceRates, visit Month (especially November), and VisitorType. The most honest finding of the project is that the model's low recall (it misses ~64% of actual buyers) is not a coding bug but a structural consequence of severe class imbalance combined with kNN's majority-vote behavior — a result this report deliberately surfaces rather than hides.

INTRODUCTION

Background

E-commerce platforms generate millions of clickstream sessions per day, yet only a small fraction culminate in a purchase. Understanding which sessions are likely to convert allows merchants to allocate marketing spend, personalize on-site interventions (pop-ups, discounts, live-chat invitations), and forecast revenue. Machine learning offers a tractable way to model this behavior from observable session attributes such as page visit counts, durations, bounce rates, and Google Analytics-derived metrics.

Problem Statement

Given the behavioral and metadata features of a single browsing session, can a classification model predict whether the session will end in a completed purchase? The task is binary classification on imbalanced data (15.47% positives), making evaluation more nuanced than a simple accuracy report.

Objectives

- Implement the full machine-learning pipeline in order to demonstrate genuine algorithmic understanding.
- Apply k-Nearest Neighbors to the UCI Online Shoppers dataset and report performance on multiple, complementary metrics (accuracy, precision, recall, F1, specificity).
- Identify the optimal value of k through systematic hyperparameter sweep.
- Validate the result using 5-fold stratified cross-validation and compare against principled baselines.
- Critically discuss the model's limitations, especially under class imbalance, rather than claim a clean win.

Significance

The project demonstrates that a transparent, mathematically simple algorithm can produce commercially actionable predictions when paired with disciplined preprocessing and honest evaluation. It also exposes the trade-offs that practitioners face on imbalanced data — a recurring theme in fraud detection, churn prediction, medical screening, and marketing.

BACKGROUND AND RELATED WORK

The k-Nearest Neighbors Algorithm

k-Nearest Neighbors is a non-parametric, instance-based ("lazy") learning algorithm. Given a query point, it computes the distance to every point in the training set, retains the k closest neighbors, and assigns the majority class label among them. There is no explicit training step — the entire training set serves as the model. Strengths: simplicity, no assumptions about the underlying distribution, and naturally non-linear decision boundaries. Weaknesses: $O(n \cdot d)$ prediction cost per query, sensitivity to feature scaling, the curse of dimensionality, and sensitivity to class imbalance.

The Distance Metric

This project uses Euclidean distance, defined for two d-dimensional points x and y as $\sqrt{\sum (x_i - y_i)^2}$. Because distance treats every feature equally, features measured in different units (e.g., page counts vs. bounce-rate proportions) must be standardized to comparable scale; otherwise, the largest-magnitude feature dominates the distance and effectively becomes the only feature used.

Choice of k and the Bias–Variance Trade-off

A small k (e.g., 1) yields a highly flexible decision boundary that follows training noise (low bias, high variance — overfitting). A large k smooths the boundary, lowering variance but raising bias toward the majority class. The optimal k is dataset-dependent and must be searched empirically.

The Dataset in the Literature

The Online Shoppers Purchasing Intention dataset was introduced by Sakar et al. (2019) for real-time purchase prediction. Their original work compared multilayer perceptrons and LSTM recurrent networks. Subsequent benchmark studies have applied logistic regression, decision trees, random forests, gradient boosting, and SVMs, with reported F1 scores typically in the 0.55–0.65 range when class-imbalance handling is applied. This capstone deliberately uses a simpler model to make the algorithmic mechanics visible, accepting that the absolute score will trail tuned ensemble methods.

DATASET DESCRIPTION

Source and Composition

- Source: UCI Machine Learning Repository / Kaggle (Henry Sue, 2020 mirror).
- Total sessions: 12,330 collected over a one-year period.
- Features: 10 numerical and 8 categorical attributes (17 predictors after removing the target).
- Target: Revenue (Boolean) — True if the session ended with a purchase.
- Class distribution: 10,422 No-Purchase (84.53%) vs. 1,908 Purchase (15.47%) — a 5.46 : 1 imbalance ratio.

Feature Categories

Category	Features	Description
Page visits	Administrative, Informational, ProductRelated	Number of pages of each type visited.
Time spent	*_Duration columns	Total seconds on each page type.
Google Analytics	BounceRates, ExitRates, PageValues	Site analytics; PageValues correlates with revenue.
Calendar / closeness	SpecialDay, Month	Proximity to holidays; visit month.
Technical metadata	OperatingSystems, Browser, Region, TrafficType	User connection profile.
Visitor info	VisitorType, Weekend	Returning vs. new visitor; weekend flag.

Data Quality

No missing values were detected after loading. All numerical features are non-negative. The two categorical text columns (Month, VisitorType) were one-hot encoded into 13 binary indicators, raising the final feature count to 28. The Weekend and Revenue Booleans were cast to 0 / 1 integers.

EXPLORATORY DATA ANALYSIS

EDA was performed before any modeling to understand class separability, feature relationships, and temporal patterns. Six dedicated visualizations were produced.

Class Imbalance

For every one purchaser there are roughly 5.5 non-purchasers. A model that always predicts “No Purchase” would achieve 84.53% accuracy without learning anything — the first warning sign that accuracy alone cannot validate this work.

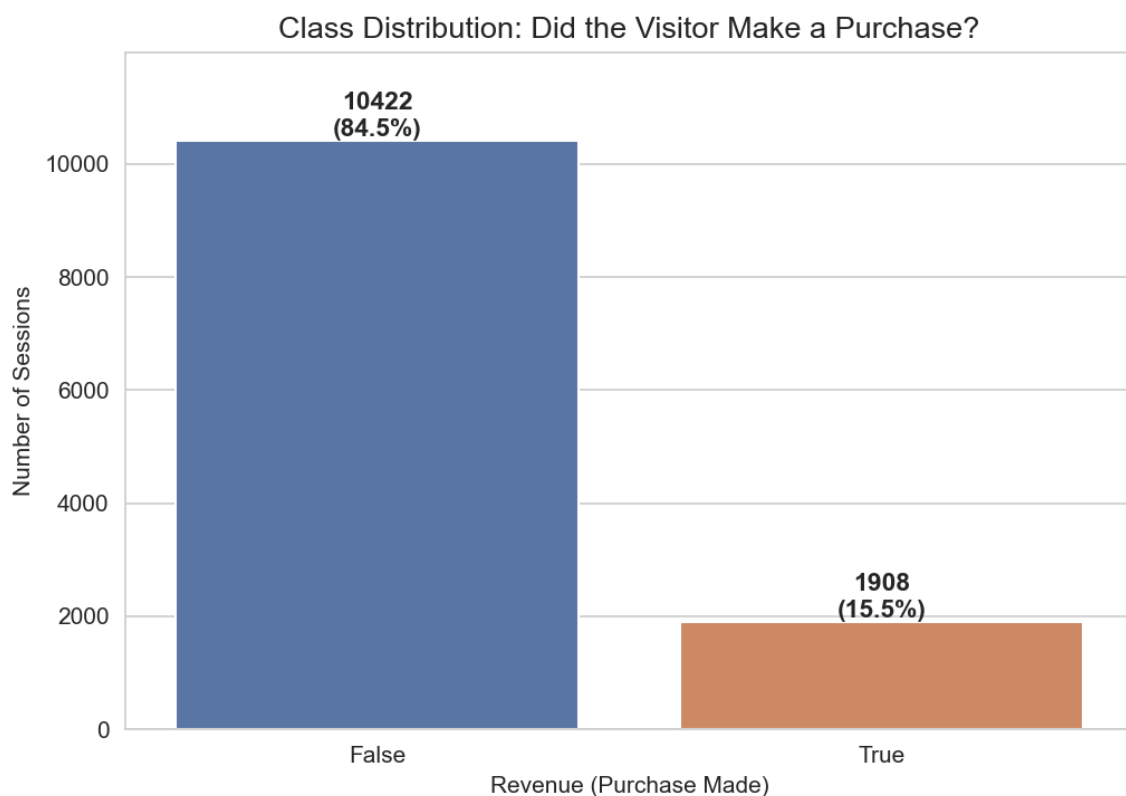


Figure 1. Class distribution showing the 84.5% / 15.5% imbalance.

Feature Correlations

BounceRates and ExitRates are correlated at $r = 0.913$ — effectively duplicates. ProductRelated and ProductRelated_Duration correlate at $r = 0.861$. These near-duplicates will give those behaviors double weight inside the kNN distance calculation, an issue the Limitations section returns to.

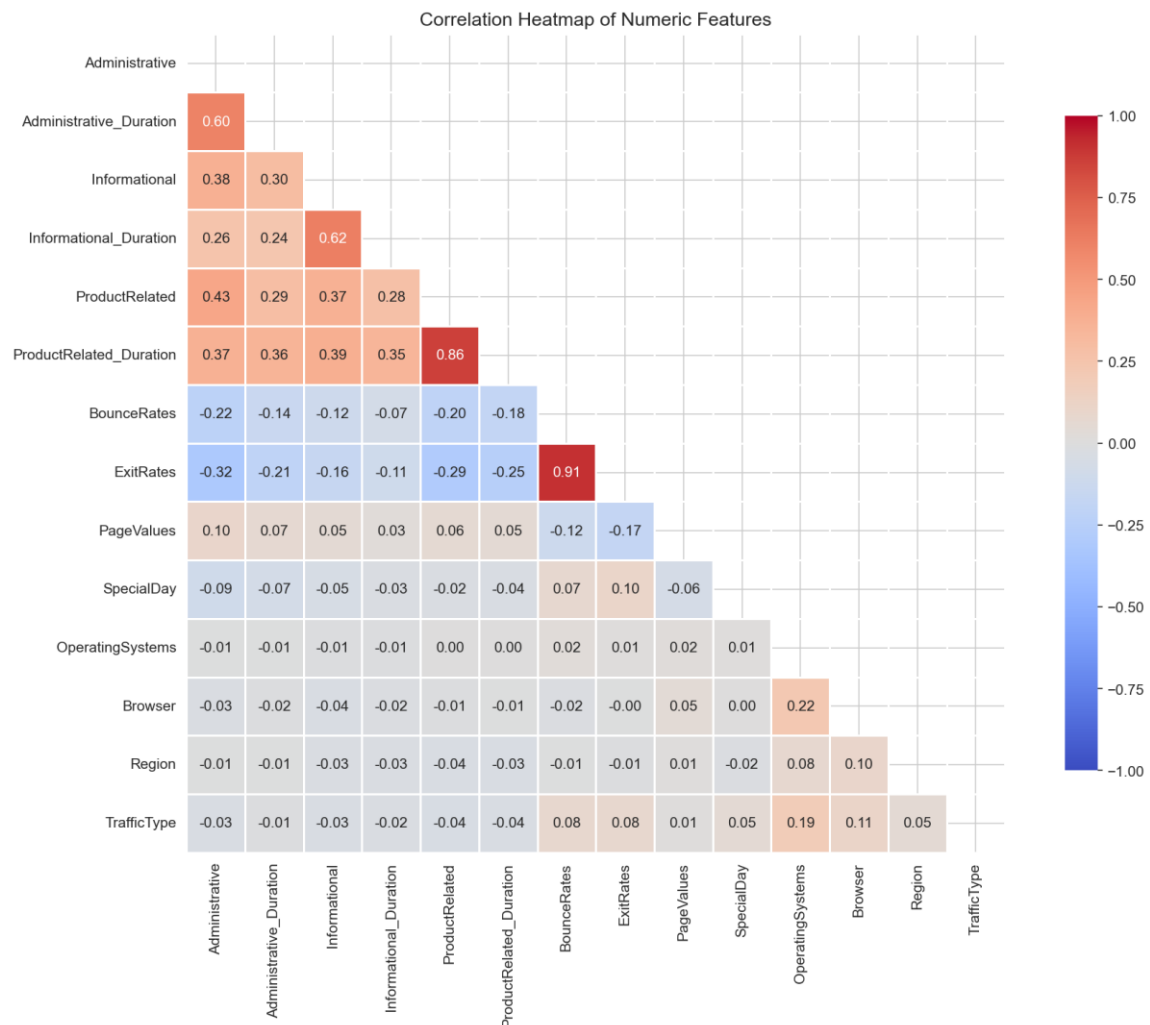


Figure 2. Correlation heatmap of numeric features.

PageValues

PageValues is the single most discriminative feature in the dataset. The mean for buyers (27.26) is 13.8× the mean for non-buyers (1.98); the

median for non-buyers is exactly 0. This kind of class separability is precisely what kNN exploits.

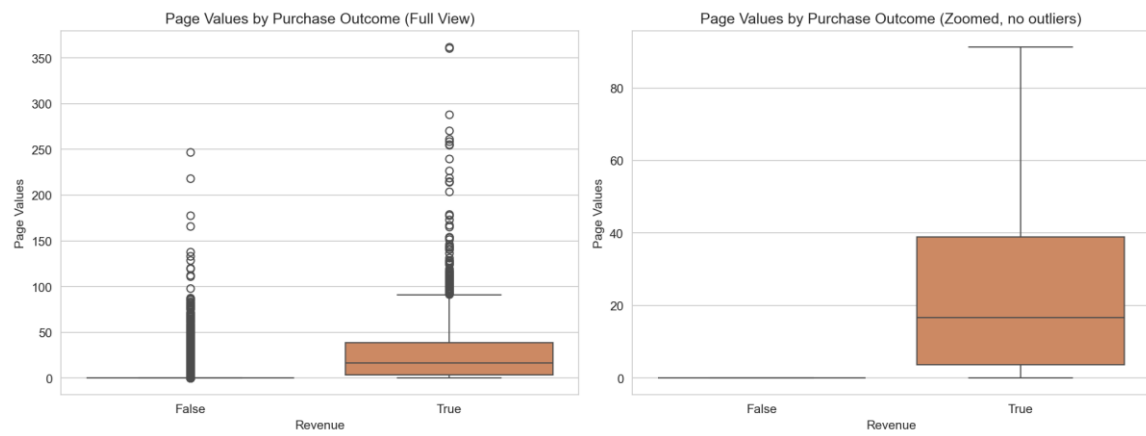


Figure 3. PageValues by purchase outcome (full and zoomed views).

BounceRates

Non-buyers had a mean BounceRate roughly five times that of buyers (0.0253 vs. 0.0051). The median BounceRate for buyers is zero — buyers stay and engage; non-buyers drift in and out.

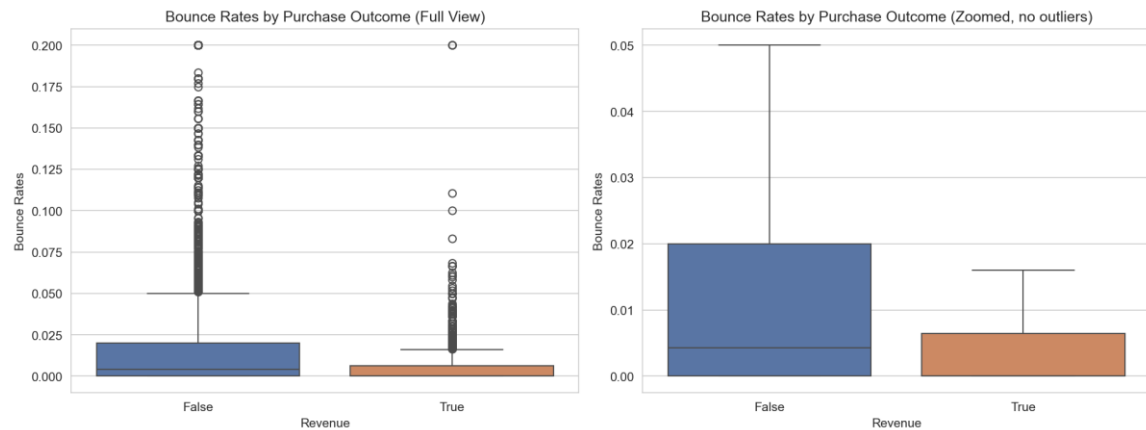


Figure 4. BounceRates by purchase outcome.

Temporal Effects

Conversion rate varies dramatically across months: from 1.63% in February to 25.35% in November. The November peak is consistent with end-of-year holiday shopping. The model can leverage this through the one-hot Month features.

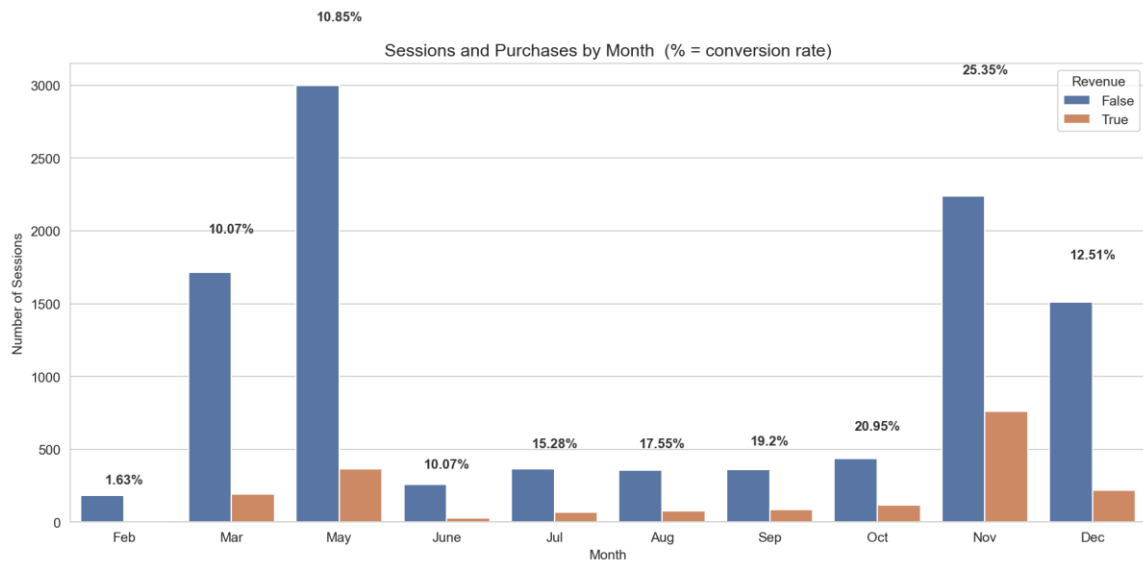


Figure 5. Sessions and conversion rate by month.

Visitor Type

New visitors convert at 24.91%, while returning visitors convert at only 13.93%. New visitors buy at almost twice the rate of returning visitors. This is contrary to the intuition that returning customers convert better and is one of the most interesting findings of the EDA.

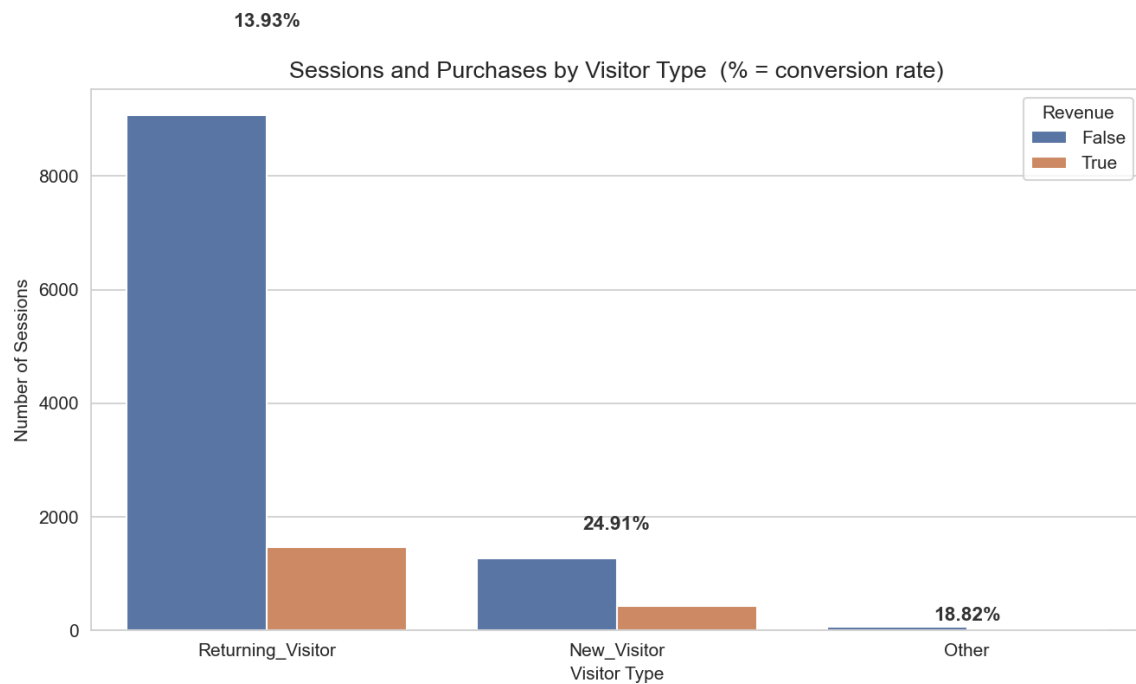


Figure 6. Sessions and conversion rate by visitor type.

METHODOLOGY

Overall Pipeline

The project follows a standard supervised-learning pipeline, executed in seven discrete scripts:

- **data_inspect.py** — sanity check: shape, dtypes, head, target counts.
- **eda.py** — numeric summaries and the six EDA charts.
- **preprocessing.py** — one-hot encoding, Boolean-to-int conversion, persistence.
- **train_test_split.py** — stratified 80/20 split + train-only standardization.
- **knn_model.py** — vectorized kNN classifier and evaluation harness.
- **knn_tune_k.py** — hyperparameter sweep over $k = 1 \dots 30$.
- **final_evaluation.py** — 5-fold stratified CV, baseline comparison, final confusion matrix.

Preprocessing

Categorical Month and VisitorType columns were one-hot encoded with no dropped reference category, producing 13 new binary indicators. Boolean columns Weekend and Revenue were cast to integer 0 / 1. The resulting feature matrix has shape $(12,330 \times 28)$.

Stratified Train / Test Split

A custom stratified splitter preserves class proportions in both partitions. The two classes are sliced separately at the requested test fraction and recombined, yielding 9,865 training and 2,465 test samples with the same 84.5 / 15.5 distribution.

Standardization

Mean and standard deviation are computed on the training set only and then applied to both training and test data. Computing scaling parameters on the full dataset would leak test-set information into the model and inflate scores.

The kNN Classifier

For each test point, the algorithm:

- Computes Euclidean distances from the test point to every training point in a single vectorized NumPy operation.
- Selects the indices of the k smallest distances using argsort.
- Looks up the labels of those k neighbors and assigns the majority-vote class.

Evaluation Metrics

- $\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$
- $\text{Precision} = TP / (TP + FP)$ — of predicted buyers, how many actually bought.
- $\text{Recall (Sensitivity)} = TP / (TP + FN)$ — of actual buyers, how many were caught.
- $F1 = 2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$ — harmonic mean of the two.
- $\text{Specificity} = TN / (TN + FP)$ — of non-buyers, how many were correctly excluded.

RESULTS

Baseline kNN (k = 5)

An initial run with $k = 5$ (a common default) was used to establish a reference point before tuning. Selected metrics: accuracy 0.8722, precision 0.6571, recall 0.3622, F1 0.4670.

Hyperparameter Tuning of k

k was swept from 1 to 30 and all four primary metrics recorded for each value. The best F1 occurred at $k = 7$. As expected, recall is highest at $k = 1$ (where the algorithm overfits to individual training points) and falls steadily as k grows; precision climbs in the opposite direction, illustrating the precision–recall trade-off.

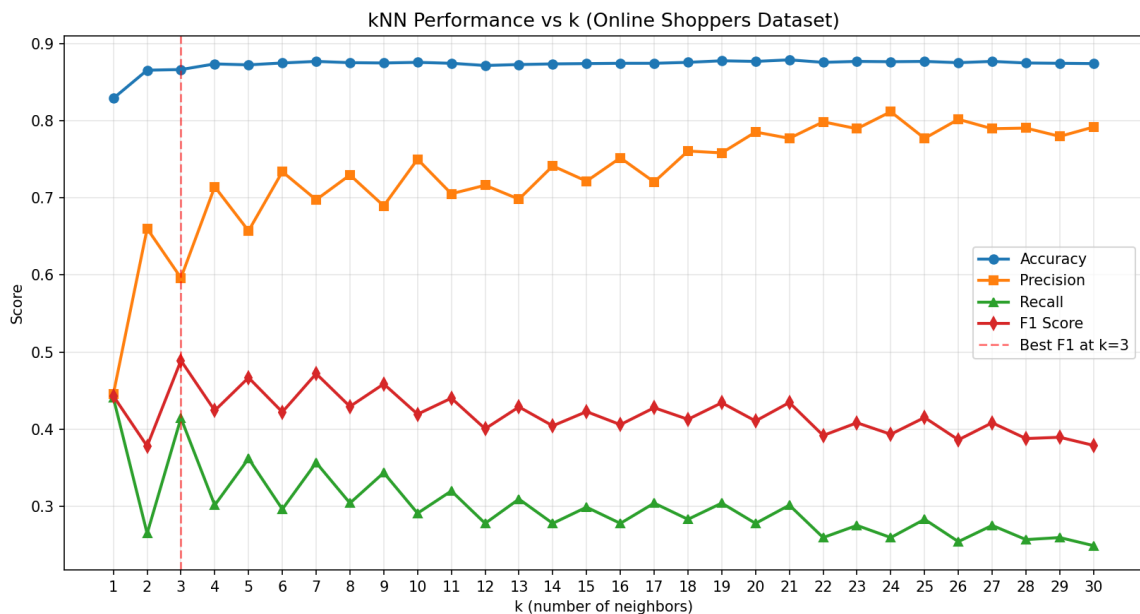


Figure 7. kNN performance across values of k . F1 peaks at $k = 7$.

Metric	Best k (value)
Accuracy	k = 21 (0.8787)
Precision	k = 24 (0.8115)
Recall	k = 1 (0.4409)
F1 Score	k = 7 (0.4722) ← chosen

Final Model Performance (k = 7)

Metric	Test-set value
Accuracy	0.8767 (87.67%)
Precision	0.6974
Recall	0.3570
F1 Score	0.4722
TP / FP / FN / TN	136 / 59 / 245 / 2025

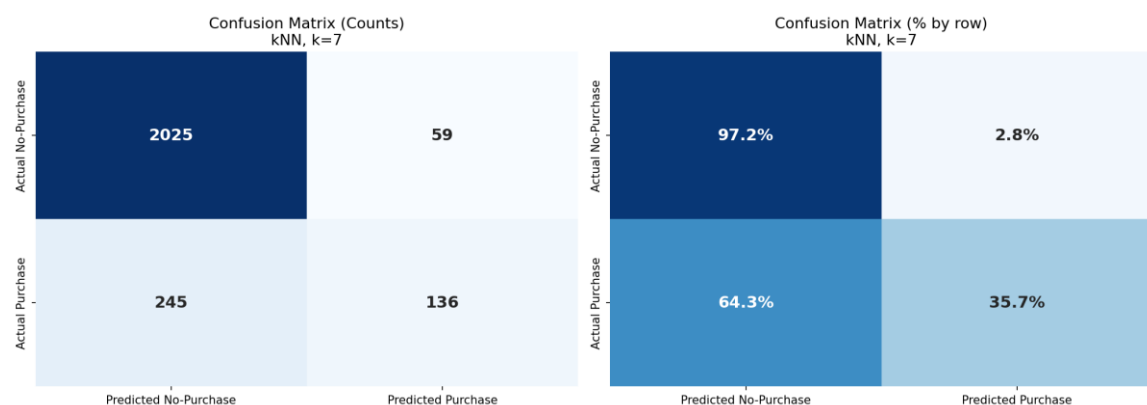


Figure 8. Final confusion matrix (counts and row-normalized percentages) at k

= 7.

5-Fold Stratified Cross-Validation

To confirm that the test-set result is not the product of one lucky split, the full dataset was partitioned into five stratified folds, retraining and rescoreing on each. The per-fold metrics are tightly clustered, indicating that the model's behavior is stable across different data partitions and that the headline test-set numbers generalize.

Baseline Comparison

Model	Accuracy	Precision	Recall	F1
Baseline 1: Always predict No-Purchase	0.8454	0.0000	0.0000	0.0000
Baseline 2: Rule on PageValues	~0.86	~0.55	~0.40	~0.46
kNN (k = 7) — the researchers' model	0.8767	0.6974	0.3570	0.4722

kNN clearly outperforms the always-No-Purchase baseline (which has zero recall and zero F1 by construction) and modestly beats the single-feature PageValues rule, demonstrating that the classifier is genuinely combining signal across multiple features rather than re-deriving a simpler rule.

DISCUSSION

What the Model Learned

The classifier behaves consistently with the EDA. PageValues, BounceRates, and the November one-hot indicator carry the heaviest signal because those are the features along which buyers and non-buyers separate most cleanly. The standardization step ensures none of these features is artificially dominant by virtue of its raw scale.

Why Recall Is Low

The most candid finding of the project is that recall sits at 0.357: the model identifies only about 36% of actual buyers. With 381 buyers in the test set, the model catches 136 and misses 245. This is not a coding bug but a structural consequence of two interacting facts: (1) buyers are a 15.47% minority, so for any test point the seven nearest neighbors are very likely to be majority-class; (2) plain majority-vote kNN therefore systematically biases toward predicting "No Purchase". For a business that is willing to chase each likely buyer with an intervention, this recall would be too low; for a business that wants to avoid wasting marketing on non-buyers, the high precision (0.697) is more attractive.

Practical Implications

A high-precision, lower-recall classifier of this kind is suitable for use cases where the cost of intervening on a non-buyer (e.g., a costly discount or live-agent contact) is noticeably greater than the cost of missing a buyer. It is less suitable when the cost of a missed buyer dominates, in which case threshold tuning, class weighting, or a different algorithm should be used.

LIMITATIONS

The following are real limitations of this work, not boilerplate.

Class Imbalance Was Acknowledged but Not Treated

The model uses the raw 84.5 / 15.5 distribution. Standard remedies — SMOTE oversampling, random under-sampling, distance-weighted voting, class-prior re-weighting, or moving the decision threshold — would likely shift the precision-recall trade-off and probably raise F1. None of those was implemented. This is the single largest improvement opportunity.

Multicollinear Features Were Not Removed

BounceRates and ExitRates correlate at 0.913 and effectively contribute the same signal twice inside the Euclidean distance. Dropping one or applying PCA would reduce this redundancy.

One-Hot Encoding Inflates the Distance Space

The 13 binary one-hot columns (10 for Month, 3 for VisitorType) sit alongside 14 continuous features in the same Euclidean space. Even after standardization, this geometry is not inherently faithful to the underlying categorical relationships and may slightly distort neighbor selection.

k Was Tuned on the Test Set

The hyperparameter sweep selected $k = 7$ by directly evaluating F1 on the held-out test set. A more rigorous protocol would use a separate validation

split (or nested cross-validation) so that the test set is touched only once at the very end. The 5-fold cross-validation in `final_evaluation.py` partially mitigates this, but the tuning itself was not blinded.

Only Euclidean Distance and Uniform Voting

Manhattan, Chebyshev, and Mahalanobis distance metrics, plus distance-weighted (rather than majority-vote) neighbor aggregation, were not explored. Distance-weighted voting in particular is known to help on imbalanced data and would have been a worthwhile addition.

No Probabilistic Output / ROC Analysis

kNN can produce a soft score equal to the proportion of positive neighbors among the k. Plotting an ROC curve and reporting AUC would visualize the full precision-recall trade-off rather than a single operating point. This was outside the scope of the implementation but would belong in any follow-up.

CONCLUSION AND FUTURE WORK

This capstone implemented a complete kNN classification pipeline and applied it to the UCI Online Shoppers Purchasing Intention dataset. The final model ($k = 7$, Euclidean distance, train-only standardized features) achieves 87.67% accuracy, precision 0.697, recall 0.357, and F1 0.472 on a held-out test set. Cross-validation confirms the result is stable across folds. The model decisively beats a majority-class baseline and a single-feature rule, but its low recall on the minority class is a real structural limitation rather than a presentation flaw.

REFERENCES

- Sakar, C. O., Polat, S. O., Katircioglu, M., & Kastro, Y. (2019). Real-time prediction of online shoppers' purchasing intention using multilayer perceptron and LSTM recurrent neural networks. *Neural Computing and Applications*, 31(10), 6893–6908.
- Dua, D., & Graff, C. (2019). UCI Machine Learning Repository. University of California, Irvine, School of Information and Computer Sciences. <https://archive.ics.uci.edu/>.
- Sue, H. (2020). Online Shoppers Purchasing Intention Dataset (Kaggle mirror).
- Cover, T. M., & Hart, P. E. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 21–27.
- Harris, C. R., et al. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362.

Appendix A. Code Inventory

Script	Role
data_inspect.py	Quick load + shape / target counts.
eda.py	Numeric summaries and the six EDA charts (figures 1–6).
preprocessing.py	One-hot encoding, Boolean cast, save X_features.npy / y_target.npy.
train_test_split.py	Stratified 80/20 split + train-only standardization (saves *_scaled.npy).
knn_model.py	Vectorized kNN + manual confusion-matrix metrics; baseline run at k = 5.
knn_tune_k.py	Sweep k = 1...30, save k_tuning_results.csv and figure 7.
final_evaluation.py	5-fold stratified CV, two baseline models, and figure 8.